The Journal of Open Source Software

PlasmaCalcs: A Consistent Interface for Python Plasma Calculations from Any Inputs

Samuel Evans [1¶], **Rattanakorn Koontaweepunya** [1], **Alexander Green** [1], **and Tess Goodman** [1]

1 Boston University Center for Space Physics, United States ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 08 September 2025
Published: unpublished

Summary

Plasma makes up the vast majority of ordinary matter in observable universe, and plasma physics drive a wide range of phenomena, from sparks to lightning bolts, stellar formation to supernovae, and planetary atmospheres to interactions in interstellar space (Connor et al., 2025; Contemporary Physics Education Project, 2011). Plasma is a state of matter involving at least some charged particles, which can interact with electric and magnetic fields. Vast amounts of research at the forefront of human knowledge involve analyzing theory and data related to plasma physics. `PlasmaCalcs` provides a consistent interface (in Python) for plasma calculations from any inputs.

Statement of need

Plasma physics contains a myriad of quantities, mathematical operations, and potential data sources. *Quantities* include: electric and magnetic field, mass, charge, density, velocity, temperature, collision frequency, magnetization parameter, Debye length, collisional mean free path, thermal speed, zeroth-order values such as equilibrium temperatures, elemental abundances, and many more. *Operations* include: spatial and temporal derivatives, vector magnitude, cross products and dot products, Fourier transforms, and interpolation from lookup tables. *Data sources* include: observations from spacecraft or telescopes; outputs from various types of simulators, including kinetic codes (e.g., EPPIC (Oppenheim & Dimant, 2004)), multifluid codes (e.g., Ebysus (Martínez-Sykora et al., 2020)), and single-fluid radiative magnetohydrodynamic (MHD) codes (e.g., Bifrost (Gudiksen et al., 2011), MURAM (Vögler et al., 2005); and any arbitrary dataset (e.g., an artificially-constructed grid of relevant physical parameters).

Complicating matters further, quantities often have different dimensionality in different data sources. For example, magnetic field is constant in EPPIC, but varies in time and space (which may itself be 2D or 3D) in Bifrost. Also, EPPIC number density varies from species to species, while single-fluid codes like Bifrost instead track the "averaged" single-fluid number density.

Software design

`PlasmaCalcs` helps to manage all of this complexity. This package provides a consistent interface for loading data from any kind of input, has many different available calculations relevant to plasma physics, supports well-labeled multidimensional arrays via xarray (Hoyer & Hamman, 2017), features many convenient capabilities for data manipulation and exploration, and contains well-documented custom-built help systems. If you use Python to analyze any data related to plasma physics, then this package is for you!

The Journal of Open Source Software

One of the biggest advantages to using `PlasmaCalcs` is that it separates data inputs from the quantities themselves. For example:

```
import PlasmaCalcs as pc
cc = pc.BifrostCalculator(...)  # or, e.g., EppicCalculator or EbysusCalculator
cc('ldebye')
```

will get the Debye length (one of the fundamental length scales in plasma physics). The user doesn't need to know that Debye length depends on number density (n) and temperature (T), figure out how to load n and T, and remember which function to plug those values into. The user also doesn't need to consider the number of species involved, nor whether n and T vary across different dimensions; the syntax to get the Debye length is the same for all kinds of input. This greatly reduces the mental load required to do analysis. Users are free to ask interesting questions such as "does my simulation resolve the Debye length?" without always needing to consider more basic questions like "how do I compute the Debye length?" at every step of the way.

Trusting computational research results often requires many sanity checks and tests, which often require computing, inspecting, and visualizing many intermediate values. Recognizing this fact has greatly influenced `PlasmaCalcs`, leading to an object-oriented design which exposes intermediate quantities and provides helpful methods to list available quantities, recall their meanings, and determine their dependencies. For example, if Debye length results look surprising and need to be checked, the natural next steps will be `cc.help('ldebye')` to see relevant docstring(s), along with `cc.tree('ldebye')` which reveals the dependence on n and T.

The design also aims to facilitate exploration and extension, while mainly only optimizing performance where it matters in real use cases. Research aiming to make new discoveries often involves spending much more time writing code than running code. `PlasmaCalcs` reduces the time it takes to write code by creating a simple interface for defining new known variables and patterns, which usually clearly resemble the scientific formulas they represent. For example, the code for $\beta = P/(|\vec{B}|^2/(2\mu_0))$ looks like `self('P') / (self('mod_B')**2 / (2 * self.u('mu0')))`, and works properly regardless of dimensionality, variability across time, and number of fluids. While this interface adds Python overhead, in real use cases the cost is negligible compared to that of array manipulations and arithmetic (which are already handled by optimized libraries like `numpy`).

`PlasmaCalcs` aims to never silently give incorrect results, but it also aims to expose all relevant options while still remaining as easy to use as possible. To achieve this, the code is designed to crash at runtime in case of any ambiguity, such as having any conflicting, missing, or otherwise ambiguous settings. However, it also picks reasonable defaults when possible; for simulation analysis this means matching the simulation physics. For example, Bifrost T might come from the ideal gas law, equation of state lookup tables, or non-local-equilibrium physics in Bifrost itself; `PlasmaCalcs` will choose whichever one matches the given Bifrost run, by default, but it still exposes the relevant setting (e.g., `cc.eos_mode='ideal'` switches to using ideal gas law). Finally, the list of all settings which could affect results are viewable at `cc.behavior`, which helps dramatically with debugging as well as discoverability.

State of the field

Existing packages help with some of these tasks, but `PlasmaCalcs` provides notable advantages:

- `plasmapy` (PlasmaPy Community et al., 2024) provides many plasma formulas, however those formulas all require the user to directly input the relevant physical quantities (e.g., `plasmapy.formulary.lengths.Debye_length` requires `T_e` and `n_e`).
- `xarray` (Hoyer & Hamman, 2017) provides a powerful interface for analyzing multidimensional arrays, and is used extensively by `PlasmaCalcs`. However, `PlasmaCalcs`

provides many methods and quantities specific to plasma physics. `PlasmaCalcs` also provides additional convenient behaviors for plotting (e.g., put "x" on the x-axis by default) and tools for creating animations (e.g., `arr.pc.image().ani()`), which greatly facilitate exploration of multidimensional data. Additionally, `PlasmaCalcs` provides convenient access to useful scientific operations missing from `xarray`, such as `arr.pc.fft()` to take the Fourier transform.

- `sunpy` (The SunPy Community et al., 2020) provides many tools relevant to solar plasma physics, but focuses primarily on acquiring, directly inspecting, and visualizing data, rather than computing a wide variety of plasma quantities.
- `helita` provides interfaces for computing quantities from a variety of input sources, including Ebysus, Bifrost, and MURAM. `PlasmaCalcs` improves upon `helita` in a variety of ways, including:
  - a more detailed class hierarchy, which reduces repetitive code and encapsulates methods more intuitively.
  - defining one function per quantity, which allows for easier introspection.
  - making fewer assumptions about the underlying data structure (e.g., data does not need to be 3D).

Research impact statement

`PlasmaCalcs` has contributed to several recent publications studying plasma instabilities (Evans et al., 2025, 2026; Koontaweepunya et al., 2024). It has also contributed to a variety of recent scientific research endeavors, some led by collaborators other than the authors of this paper, with some manuscripts currently under review or being prepared for publication. These endeavors include:

- Analyzing a suite of EPPIC simulations across the Sun's chromosphere (under review, preprint available: Evans, Oppenheim, Martínez-Sykora, & Green, 2026).
- Analyzing plasma instabilities in a large 3D EPPIC simulation modeling a column of the Earth's ionosphere (under review, led by Meers Oppenheim).
- Comparing that 3D simulation to multiple 2D simulations to quantify the amount of coupling across altitudes (in prep., led by Rattanakorn Koontaweepunya).
- Studying the electron thermal instability in the ionosphere using the new kinetic code, COPAPIC (in prep., led by Alexander Green).
- Using outputs from the Jovian Atmospheric Model Making EStimates (JAMMIES) simulator (Moore et al., 2026) to determine the importance of similar plasma instabilities in Jupiter's ionosphere (led by Samuel Evans).
- Draping 3D plane-parallel Bifrost and MURAM simulations onto the Sun while respecting solar curvature, interpolating quantities onto computed ray paths, and integrating in order to synthesize observables for the NASA MUSE mission (led by Juan Martínez-Sykora and others collaborating on MUSE).
- Constructing hydrostatic and ionization equilibria for multi-fluid-multi-species plasma modeling of the lower solar atmosphere and analyzing corresponding simulations (led by Fan Zhang).

Additionally, work is underway to set up `PlasmaCalcs` for analyzing outputs from the Multiscale Atmosphere-Geospace Environment (MAGE) simulator (Pham et al., 2022; Sorathia et al., 2023), developed by the Center for Geospace Storms. MAGE brings together multiple models, including: Grid Agnostic MHD for Extended Research Applications (GAMERA) for the magnetosphere at large (Zhang et al., 2019), the Rice Convection Model (RCM) for the ring current in the inner magnetosphere (Toffoletto et al., 2003), and a rewrite of the Magnetosphere-Ionosphere Coupler/Solver (MIX) code for the ionosphere (Merkin & Lyon, 2010). While the open-source package `kaipy` provides some analysis routines for MAGE outputs, `PlasmaCalcs` may provide notable benefits thanks to its consistent interface across models, higher-dimensional visualization routines, and well-labeled arrays which directly include

information about the corresponding coordinates.

AI usage disclosure

This work used generative AI tools as follows. GitHub Copilot was used to provide inline autocomplete suggestions while coding, often filling in pieces of code which follow naturally from nearby code or from human-written docstrings. ChatGPT was sometimes used to get code suggestions or provide suggestions while debugging, especially when first setting up a new tool or a new kind of input. All AI-generated code suggestions were made in small pieces (usually only a few lines, always or almost always less than 100 lines) and fully considered by humans. Code documentation was written primarily by human authors, with GitHub Copilot autocomplete suggestions being incorporated sometimes. Continuous integration tests, investigations into the self-consistency of research results produced by the code, and numerous spot checks conducted "by hand" all help to confirm the correctness of the code. This paper was written by human authors, with GitHub Copilot autocomplete suggestions visible but never (or almost never) incorporated, although even the visibility of these suggestions may have been enough to have a minor influence on the final text (see, e.g., Williams-Ceci et al., 2026).

Acknowledgements

We acknowledge Meers Oppenheim, Juan Martínez-Sykora, and Yakov Dimant, for many helpful conversations which contributed to our understanding of plasma physics and informed some of our choices for which plasma physics quantities to implement while developing this project. We further recognize Juan Martínez-Sykora for making contributions to `PlasmaCalcs` after this paper was originally submitted. We also acknowledge the developers of the `helita` package, which inspired various aspects of the `PlasmaCalcs` code design.

References

Connor, L., Ravi, V., Sharma, K., Ocker, S. K., Faber, J., Hallinan, G., Harnach, C., Hellbourg, G., Hobbs, R., Hodge, D., Hodges, M., Kosogorov, N., Lamb, J., Law, C., Rasmussen, P., Sherman, M., Somalwar, J., Weinreb, S., Woody, D., & Konietzka, R. M. (2025). A gas-rich cosmic web revealed by the partitioning of the missing baryons. *Nature Astronomy*, *9*, 1226–1239. https://doi.org/10.1038/s41550-025-02566-y

Contemporary Physics Education Project. (2011). *Fusion: Physics of a Fundamental Energy Source*. https://newsite.cpepphysics.org/elementor-1179/

Evans, S., Oppenheim, M., Martínez-Sykora, J., & Dimant, Y. (2025). Multifluid and Kinetic 2D and 3D Simulations of Thermal Farley–Buneman Instability Turbulence in the Solar Chromosphere. *986*(1), 23. https://doi.org/10.3847/1538-4357/adcd70

Evans, S., Oppenheim, M., Martínez-Sykora, J., & Dimant, Y. (2026). The Unstable Chromosphere: Predicting Thermal Farley Buneman Instability Growth Across a Broad Range of Solar Chromospheric Conditions. *1001*(1), 10. https://doi.org/10.3847/1538-4357/ae4ddd

Evans, S., Oppenheim, M., Martínez-Sykora, J., & Green, A. (2026). The Unstable Chromosphere: Effects of the Thermal Farley-Buneman Instability Across a Broad Range of Solar Chromospheric Conditions. *arXiv e-Prints*, arXiv:2604.25027. https://doi.org/10.48550/arXiv.2604.25027

Gudiksen, B. V., Carlsson, M., Hansteen, V. H., Hayek, W., Leenaarts, J., & Martínez-Sykora, J. (2011). The stellar atmosphere simulation code Bifrost. Code description and validation. *Astronomy & Astrophysics*, *531*, A154. https://doi.org/10.1051/0004-6361/201116520

Hoyer, S., & Hamman, J. (2017). Xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, *5*(1). https://doi.org/10.5334/jors.148

Koontaweepunya, R., Dimant, Y. S., & Oppenheim, M. M. (2024). Non-Maxwellian ion distribution in the equatorial and auroral electrojets. *Frontiers in Astronomy and Space Sciences*, *11*, 1478536. https://doi.org/10.3389/fspas.2024.1478536

Martínez-Sykora, J., Szydlarski, M., Hansteen, V. H., & De Pontieu, B. (2020). On the Velocity Drift between Ions in the Solar Atmosphere. *The Astrophysical Journal*, *900*(2), 101. https://doi.org/10.3847/1538-4357/ababa3

Merkin, V. G., & Lyon, J. G. (2010). Effects of the low-latitude ionospheric boundary condition on the global magnetosphere. *Journal of Geophysical Research: Space Physics*, *115*(A10). https://doi.org/https://doi.org/10.1029/2010JA015461

Moore, L., Melin, H., Stallard, T., O'Donoghue, J., Roberts, K., Tiranti, P. I., Mohamed, K., Agiwal, O., Müller-Wodarg, I., Knowles, K. L., Withers, P., Coffin, D., Wang, R., & Schmidt, C. (2026). Photochemistry in Jupiter's Ionosphere: Insights from Simultaneous H3+ and Electron Density Observations during Juno Perijove 54. *7*(1), 25. https://doi.org/10.3847/PSJ/ae2f49

Oppenheim, M. M., & Dimant, Y. S. (2004). Ion thermal effects on E-region instabilities: 2D kinetic simulations. *Journal of Atmospheric and Solar-Terrestrial Physics*, *66*(17), 1655–1668. https://doi.org/10.1016/j.jastp.2004.07.007

Pham, K. H., Zhang, B., Sorathia, K., Dang, T., Wang, W., Merkin, V., Liu, H., Lin, D., Wiltberger, M., Lei, J., Bao, S., Garretson, J., Toffoletto, F., Michael, A., & Lyon, J. (2022). Thermospheric density perturbations produced by traveling atmospheric disturbances during august 2005 storm. *Journal of Geophysical Research: Space Physics*, *127*(2), e2021JA030071. https://doi.org/https://doi.org/10.1029/2021JA030071

PlasmaPy Community, Murphy, N. A., Everson, E. T., Stańczak-Marikin, D., Heuer, P. V., Kozlowski, P. M., Johnson, E. T., Malhotra, R., Schaffner, D. A., Vincena, S. T., Abler, M., Addison, J., Ahamed, A. F., Alarcon, P. V., Antognetti, B., Arran, C., Bagherianlemraski, H., Beckers, J., Bedmutha, M., … Zhang, C. (2024). *PlasmaPy* (Version v2024.10.0). Zenodo. https://doi.org/10.5281/zenodo.14010450

Sorathia, K. A., Michael, A., Merkin, V. G., Ohtani, S., Keesee, A. M., Sciola, A., Lin, D., Garretson, J., Ukhorskiy, A. Y., Bao, S., Roedig, C. B., & Pulkkinen, A. (2023). Multiscale magnetosphere-ionosphere coupling during stormtime: A case study of the dawnside current wedge. *Journal of Geophysical Research: Space Physics*, *128*(11), e2023JA031594. https://doi.org/https://doi.org/10.1029/2023JA031594

The SunPy Community, Barnes, W. T., Bobra, M. G., Christe, S. D., Freij, N., Hayes, L. A., Ireland, J., Mumford, S., Perez-Suarez, D., Ryan, D. F., Shih, A. Y., Chanda, P., Glogowski, K., Hewett, R., Hughitt, V. K., Hill, A., Hiware, K., Inglis, A., Kirk, M. S. F., … Dang, T. K. (2020). The SunPy Project: Open Source Development and Status of the Version 1.0 Core Package. *The Astrophysical Journal*, *890*, 68–68. https://doi.org/10.3847/1538-4357/ab4f7a

Toffoletto, F., Sazykin, S., Spiro, R., & Wolf, R. (2003). Inner magnetospheric modeling with the Rice Convection Model. *107*(1), 175–196. https://doi.org/10.1023/A:1025532008047

Vögler, A., Shelyag, S., Schüssler, M., Cattaneo, F., Emonet, T., & Linde, T. (2005). Simulations of magneto-convection in the solar photosphere. Equations, methods, and results of the MURaM code. *Astronomy & Astrophysics*, *429*, 335–351. https://doi.org/10.1051/0004-6361:20041507

Williams-Ceci, S., Jakesch, M., Bhat, A., Kadoma, K., Zalmanson, L., & Naaman, M. (2026). Biased AI writing assistants shift users' attitudes on societal issues. *Science Advances*, *12*(11), eadw5578. https://doi.org/10.1126/sciadv.adw5578

Zhang, B., Sorathia, K. A., Lyon, J. G., Merkin, V. G., Garretson, J. S., & Wiltberger, M. (2019). GAMERA: A three-dimensional finite-volume MHD solver for non-orthogonal curvilinear geometries. *The Astrophysical Journal Supplement Series*, *244*(1), 20. https://doi.org/10.3847/1538-4365/ab3a4c